

Playing Mastermind with AI

Chongwen Zhao, Jesen Tanadi, Junxian Zhu, Steven Yang
Duke University ME 555: Introduction to Robotics and Automation, Fall 2025

Documentation of Robot Actions

Mastermind is a code-breaking game for two players invented in Israel. In the game, one player acts as a codesetter, and another acts as a codebreaker. The codesetter's job is to decide on a sequence of colors (i.e., a secret code), while the codebreaker needs to guess this sequence of colors based on the feedback of the codesetter. The codebreaker wins by identifying the exact color sequence within a limited number of rounds; if they fail to do so, the codesetter wins the game.

In this project, we implement a robotic system that plays the Mastermind board game against a human player. In our implementation, the human player is the codesetter and the robotic system is the codebreaker. Colored blocks are used as the game tokens, and the robot physically selects and arranges these blocks in the detection area to form its guesses during gameplay.

Our system consists of a Kinova Gen3 Lite robotic arm, a Logitech RGB web camera, and a Large Language Model (LLM) agent (ChatGPT). The physical game pieces are color-coded 3D-printed blocks: red, blue, yellow, purple, black, and green.

The entire system's workflow is as follows: to initialize the game, a human player inputs a sequence of four colors (e.g., "red blue black yellow"); this sequence is stored in the game's state system. Using ChatGPT via its API, the system proposes a sequence of colors and publishes the guess to the `/submit_code` topic. This event triggers the robotic arm, which then finds the specified color-coded blocks from a set location, picks them up, and places them on a staging area. Once the pick-and-place sequence has been completed, the arm updates the game's status by publishing to the `/game_status` topic. This, in turn, triggers the computer vision system, which uses the camera to obtain a frame, and using OpenCV, segments and detects the colors of the chosen blocks. Finally, the game system checks the correctness of the code and feeds this information back to the ChatGPT agent, so that it is able to decide on the next sequence of colors to try. Between rounds, a human operator returns the blocks to their initial positions, and the simulation mirrors this action using a reset script.

For the design of the system, the entire game system consists of five ROS 2 nodes: a Mastermind node that acts as the game's entry point and activates all other nodes; a GameState node that acts as the game's "backend," keeping track of the game's state and passes messages appropriately; a Player2 node that interfaces with the ChatGPT API, making

guesses during the game; a PickAndPlace node that controls the Kinova Gen3 Lite robotic arm and places the correct blocks in the staging area; and a Vision node that interfaces with the camera and the computer vision system, scanning the colors of the game pieces in the staging area. Figure 1 below shows these nodes and their communication pathways. In addition to these nodes, a set of custom messages (Code, Status, GuessCheck) was implemented. We further discuss these components in the Code Explanation section; the code is available on [GitHub](#).

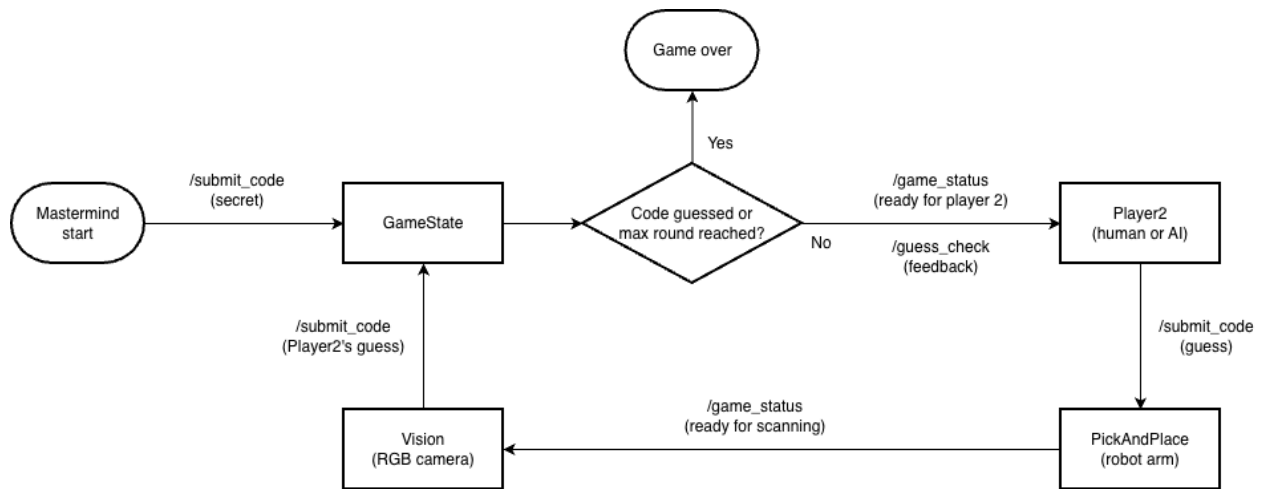


Figure 1: System diagram of the game's nodes, overall gameplay loop, and communication channels (topics) between the nodes.

Demonstration: Simulation

In this section, we show screenshots from the Gazebo simulation.

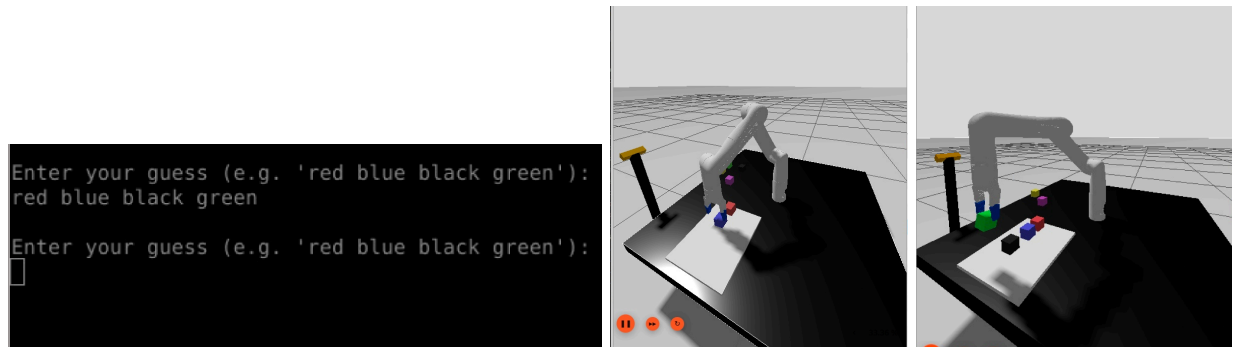


Figure 2: Player 2 enters the guess “red blue black green” and the robot arm responds by placing the corresponding colored blocks onto the staging area.

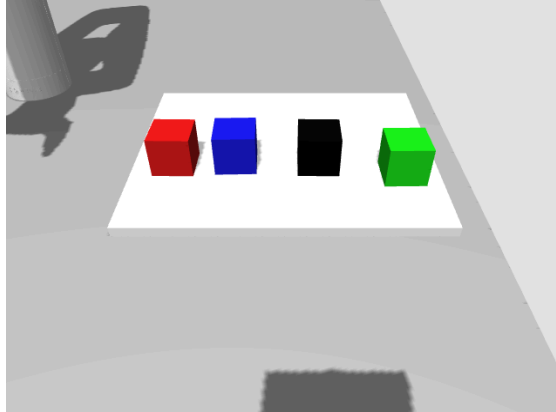


Figure 3: Simulated camera view showing game pieces in the staging area.

Demonstration: Physical Implementation

In this section, we show stills and screenshots from the physical implementation.

```
colcon build [2/2 done] [0 ongoing] 70x19
kinova [ws:kortex_ws]:/root/workspaces/mastermind_ws# ros2 run mastermind player2 --ros-args -p mode:=ai
[INFO] [1765057101.758636346] [player_2]: Player 2 running in AI mode.
[INFO] [1765057111.863649114] [player_2]: Initial guess uses four distinct colors to cover half of the color set, maximizing information under the no-repetition rule and keeping many possibilities open for the secret.
[INFO] [1765057111.865444473] [player_2]: We are going to guess list ['red', 'green', 'blue', 'yellow']
```

Figure 4: Still from screencast showing the ChatGPT agent making a guess.

```
colcon build [2/2 done] [0 ongoing] 70x19
kinova [ws:kortex_ws]:/root/workspaces/mastermind_ws# ros2 run mastermind player2 --ros-args -p mode:=human
[INFO] [1765056855.736347342] [player_2]: Player 2 running in HUMAN mode.

Enter your guess (e.g. 'red blue black green'):
blue red green yellow
```

Figure 5: Still from screencast showing a human player entering their guess in the terminal.

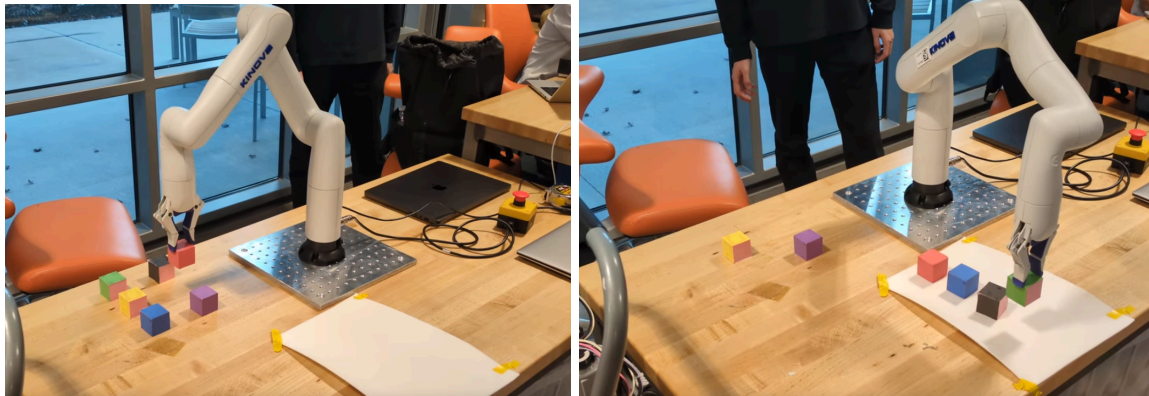


Figure 6: Still images from video recording of the pick and place sequence.

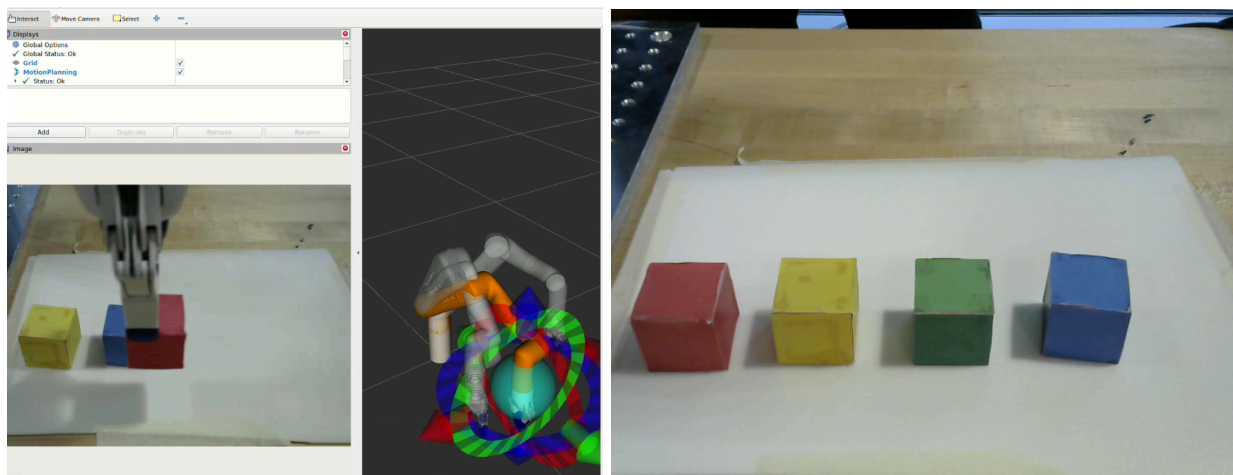


Figure 7: A still image from an RViz screencast showing the camera view and the final camera capture that is used by the computer vision model to determine color sequence.

Contributions

Below we list each team member's specific contributions. In practice, there were substantial overlaps, and our workflow was fairly collaborative, especially as we worked towards completing the physical implementation of the game.

Chongwen Zhao Developed the Player2 node and implemented communication with the ChatGPT API. Contributed to final system-wide integration in simulation and on the physical robot, and ensured that the game's message formatting and guess-handling works reliably across all components.

Jesen Tanadi Created the GameState node and designed the project's custom message types and communication structure. Set up the overall messaging framework (topics, publishers, and subscribers) and supported simulation and physical integration of the game's components.

Junxian Zhu	Developed the PickAndPlace node and prepared the physical and simulated environment. Contributed to hardware-related tasks such as setting up the workspace and 3D-printing the game pieces used during testing and demonstrations.
Steven Yang	Implemented the Vision node and integrated the camera with the robotic arm. Calibrated the vision pipeline by tuning color-detection thresholds to ensure accurate piece recognition. Contributed to perception reliability and smooth interaction between the camera and the rest of the system.

Code Explanation

The code is organized into two primary directories: **mastermind** and **mastermind_interfaces**. The first package consists of the main nodes of the game; the second package consists of the three custom messages used. Below, we document the messages and nodes. For each node, we highlight the class's core method and give a brief description of its functionality. The full code can be inspected on [GitHub](#).

Directory Structure

Below is the directory structure used in this project. To ensure that our code remains clean and organized, top-level directories are treated as separate packages. This means that while mastermind has to import messages from the mastermind_interfaces package, any changes made in one package doesn't require the entire system to be rebuilt and reinstalled.

```

src/
├── mastermind/mastermind/src/
│   ├── main.py
│   ├── game_state/
│   ├── players/
│   ├── pick_and_place/
│   └── vision/
└── mastermind_interfaces/msg/
    ├── Code.msg
    ├── GuessCheck.msg
    └── Status.msg

```

Custom messages

Code.msg

This message is used to communicate both the secret code chosen by the human and the guesses generated by the LLM agent. It contains a list of four colors (or their encoded integer representations), providing a consistent format for transmitting color sequences between nodes.

```
string player_name
uint8[4] code
```

GuessCheck.msg

This message provides feedback on each guess by reporting how many colors are correct and how many positions are correct. It is used by the GameState node to evaluate a guess and by the Player2 node (LLM agent) to determine the next guess.

```
int8 num_correct_colors
int8 num_correct_pos
```

Status.msg

This message indicates which component the system is currently waiting on (i.e., the game's status). Nodes publish a status update whenever they complete their task, enabling the GameState node to coordinate the sequence of actions across the entire system.

```
string sender
uint8 status
```

Game Nodes

Mastermind (src/main.py)

Acts as the system's entry point. This node initializes and coordinates the entire game: startup, triggering guesses, managing round flow, and orchestrating the activation of other nodes. It ensures that the game progresses properly from human input (codesetter) to guess (by codebreaker/LLM) to pick-and-place (robot arm) to vision feedback to evaluation and then finally to next guess.

main()

This method initializes ROS2 and enters the main loop to manage game rounds. This function ensures that all other nodes are correctly spun up and that message flow begins.

GameState (src/game_state/game_state.py)

Acts as the "backend" logic of the game. It keeps track of the secret code (provided by the human), records guesses, stores feedback, and determines when the game ends (win or lose). It also forwards messages appropriately between Player2, PickAndPlace, and Vision.

handle_code(msg)

This callback function handles incoming guess messages (Code.msg) from players and the computer vision system. It receives the guess, stores it, and then issues commands

to trigger the next appropriate node. It also checks whether the guess is final (i.e. correct or maximum rounds reached).

Player2 (src/players/player2.py)

Interfaces with the LLM (ChatGPT) via its API. Given feedback from previous guesses (via `GuessCheck.msg`) and the history of guesses, this node constructs the next guess by prompting the LLM, then publishes that guess to the game's messaging system (on the appropriate topic).

guess()

This function sends a request to the ChatGPT API with the conversation context and previous feedback, retrieves the LLM's response (the next color sequence), validates or formats it into the structure required by `Code.msg`, and publishes it to the `/submit_code` topic.

PickAndPlace (src/robot_controller/pick_and_place.py)

Controls the physical robot arm. When a guess is submitted, this node reads the requested color sequence, picks the corresponding 3D-printed blocks from the storage area, and places them in the staging area so that they can be visually inspected. It handles motion planning, grasping, and arm coordination.

task_place_by_color(order_str)

Given a list of colors (or encoded color IDs), this function computes the grasp positions for each block, issues arm motion commands (pick up, move, and place), monitors execution status, and once placement is done, publishes a status update so the Vision node can begin scanning.

Vision (src/vision_model/vision.py)

Integrates with the camera and runs the computer vision pipeline (via OpenCV). After the robot arm places blocks on the staging area, this node captures an image, recognizes the colors of the placed blocks, and produces a list of indices corresponding to those colors.

process_frame()

This method captures the current camera frame, crops it to the relevant region, and applies calibrated hue, saturation, and value thresholds to generate color masks. From these masks, the corresponding block regions are extracted and labeled. The resulting color labels are then sorted in the expected left-to-right order and published as a `Code.msg` containing the indices of the colors placed in the staging area for the `GameState` node.

Human Integration

In addition to the human player (codesetter), the current physical implementation requires a human to assist in resetting the staging area and the box storage area after each guess. To maintain safety, we pause the game for a short period of time (30 seconds) after the computer vision system checks the placed blocks against. The flow diagram below illustrates this short delay.



Figure 8: Flow diagram showing the delay (in red) between the game sequence to allow for human intervention.

Challenges

Throughout the project, one of our main challenges was organizing the system into clear, functional components and determining how information should flow between them. With multiple nodes handled by different team members, integration proved difficult, especially since topics, custom messages, and timing all needed to align consistently.

Vision-related issues also posed significant challenges: the camera was initially difficult to interface reliably with the robot environment, and early segmentation results were highly sensitive to lighting variations and incorrect HSV thresholds. Achieving dependable color classification required careful calibration, repeated experimentation with mask parameters, and systematic testing under different lighting conditions. On the hardware side, getting the robot arm to perform physically consistent pick-and-place actions demanded tuning motion parameters, adjusting grasp positions, and iterating through many trial-and-error cycles.

We overcame these issues by introducing custom messages and refining our ROS communication structure, performing repeated end-to-end tests, calibrating the vision pipeline to ensure stable segmentation, and incrementally adjusting both perception and control components until all subsystems interacted smoothly.

Future Work

Given additional time, one improvement to consider would be the human safety factor. Currently a short delay is hard-coded between rounds so that a human can assist in resetting the taskspace. Since the system already employs a camera, a natural improvement would be to pause the game whenever a human enters the camera's field of view, within the volume of the taskspace. This would allow for the human agent to continuously deal with errors during the entirety of the game without safety concerns.

Takeaways

Clear communication structures are essential for multi-node ROS systems.

One of the biggest lessons from this project was that a robotic system composed of many independent nodes requires a carefully planned communication architecture. Defining consistent message types, establishing predictable topic flows, and ensuring synchronized timing allowed the system to behave coherently. Future projects will benefit from designing the messaging framework early and revisiting it frequently during integration.

Incremental integration is far more effective than integrating everything at once.

Because each team member worked on a separate component, early assumptions often changed as modules were connected. We learned that integrating nodes in small steps (testing message flow, verifying behavior, and adjusting parameters) prevents large system-wide failures later. Building a habit of early and continuous end-to-end testing will make future robot programming tasks significantly smoother.

Hardware reliability depends heavily on calibration and physical iteration.

Both the camera system and the robot arm required repeated tuning to behave predictably. Color thresholds, lighting conditions, grasp positions, and motion parameters all needed iterative refinement before the system became reliable. This reinforced the idea that real-world robotic systems rarely work “out of the box,” and that deliberate calibration is just as important as clean code.

Appendix: Relevant Code

mastermind_interfaces/msg/Code.msg

```
# Generic message to send code. Used in /submit_code and /submit_guess

string player_name  # name of the player submitting the guess
uint8[4] code       # array of numerical value of colors (e.g., [1, 2, 1, 4])
```

mastermind_interfaces/msg/GuessCheck.msg

```
# Keep track of number of correct colors and their correct positions

int8 num_correct_colors # number of correct colors
int8 num_correct_pos    # number of correct positions
```

mastermind_interfaces/msg/Status.msg

```
# Generic status message

string sender  # Status sender
uint8 status   # Some pre-defined status. See node and topics document.
```

mastermind/src/main.py

```
#!/opt/ros_python/bin/python3

import os
import subprocess
import threading
import time
from typing import List

import rclpy
from mastermind_interfaces.msg import Code, Status
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from .game_state.game_state import COLOR_TO_NUM, GameState
from .robot_controller.pick_and_place import PickAndPlaceNode
from .vision_model.vision import VisionNode

class Mastermind(Node):
    """
    This node acts like Player 1 and runs everything else.
    """

    def __init__(self):
        super().__init__("mastermind")
        self.declare_parameter("secret", "")
        self.declare_parameter("sim", False)

        # Game nodes
        self.game_state = GameState()
        self.pick_and_place = PickAndPlaceNode()
        self.vision = VisionNode()

        # Pub/subs
        self.code_pub = self.create_publisher(Code, "submit_code", 10)
        self.game_status_sub = self.create_subscription(
            Status, "game_status", self.handle_status, 10
        )

        self.game_in_progress = False
        self.sim_mode = self.get_parameter("sim").get_parameter_value().bool_value

    def handle_status(self, msg: Status):
        sender = msg.sender
        status = msg.status
```

```

self.get_logger().info(f"Sender {sender} status {status}")

if sender == "game_state" and status == 0 and self.game_in_progress:
    if self.sim_mode:
        # Call a script to reset blocks in Gazebo
        self.reset_blocks()

    self.pick_and_place.reset_blocks()

def check_secret(self, secret):
    if not secret:
        raise ValueError(f"No secret provided!")

    secret_list = [s for s in secret.split() if s]
    if len(secret_list) != len(set(secret_list)):
        raise ValueError(f"Secret must contain 4 different colors!")

    allowed = {"blue", "yellow", "green", "red", "purple", "black"}
    invalid = [c for c in secret_list if c not in allowed]
    if invalid:
        raise ValueError(
            "Colors must be four of this list: 'blue', 'yellow', 'green',
            'red', 'purple', 'black'"
        )
    return secret_list

def reset_blocks(self):
    script_path = (
        "/root/workspaces/mastermind_ws/src/mastermind/world/reset_blocks.sh"
    )
    subprocess.run(["bash", script_path], check=True)

def run(self):
    secret = self.get_parameter("secret").get_parameter_value().string_value

    secret_list = self.check_secret(secret)

    executor = MultiThreadedExecutor(num_threads=8)
    executor.add_node(self)
    executor.add_node(self.game_state)
    executor.add_node(self.pick_and_place)
    executor.add_node(self.vision)

    try:
        spin_thread = threading.Thread(target=executor.spin, daemon=True)
        spin_thread.start()
        self.get_logger().info("Adding scene...")

```

```

self.pick_and_place.add_scene()
time.sleep(3)
self.get_logger().info("Adding scene completed")

# Wait until we have enough subscribers to submit_code
# (i.e., that GameState is active)
num_subs_submit_code = 1
while self.code_pub.get_subscription_count() < num_subs_submit_code:
    self.get_logger().info("Waiting for /submit_code subscriber...")
    time.sleep(0.1)

code = [COLOR_TO_NUM[c] for c in secret_list]
self.publish_code(code)

self.game_in_progress = True
self.get_logger().info("Mastermind running. Press Ctrl+C to exit.")
while rclpy.ok():
    time.sleep(0.1)

except KeyboardInterrupt:
    self.get_logger().info("KeyboardInterrupt received. Shutting down...")
finally:
    executor.shutdown()
    spin_thread.join(timeout=1.0)
    self.game_state.destroy_node()
    self.pick_and_place.destroy_node()
    self.vision.destroy_node()
    self.destroy_node()
    rclpy.shutdown()

def publish_code(self, code: List[int]):
    """
    Publish secret code as Player 1
    """
    msg = Code()
    msg.player_name = "player_1" # player_1, player_2, computer_vision, etc.
    msg.code = code

    self.code_pub.publish(msg)
    self.get_logger().info(f"Player 1 published secret {code}!")

def start_camera_bridge(logger):
    """
    bridge Gazebo & ROS2

    ros2 run ros_gz_bridge parameter_bridge \

```

```

        "/mastermind/camera/image_raw@sensor_msgs/msg/Image@gz.msgs.Image"

"""
cmd = [
    "ros2",
    "run",
    "ros_gz_bridge",
    "parameter_bridge",
    "/mastermind/camera/image_raw" "@sensor_msgs/msg/Image" "@gz.msgs.Image",
]
logger.info("bridging images...")

proc = subprocess.Popen(
    cmd,
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)
return proc

def main():
    rclpy.init()
    mastermind = Mastermind()
    bridge_proc = start_camera_bridge(mastermind.get_logger())
    try:
        mastermind.run()
    finally:
        if bridge_proc is not None:
            try:
                mastermind.get_logger().info("Stopping camera bridge...")
                bridge_proc.terminate()
                bridge_proc.wait(timeout=2.0)
            except Exception:
                bridge_proc.kill()

    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

mastermind/src/game_state/game_state.py

```
import os
import threading
import time
from typing import List

import rclpy
from mastermind_interfaces.msg import Code, GuessCheck, Status
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

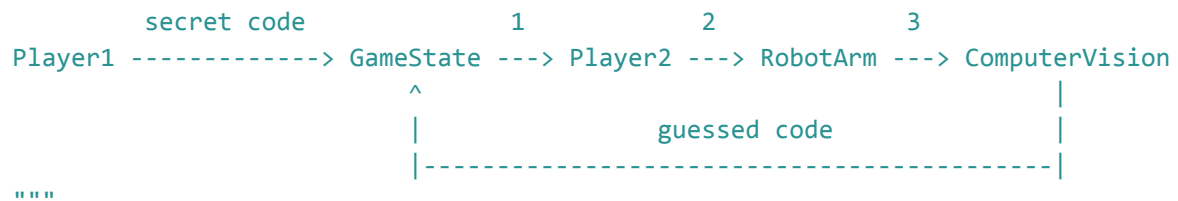
NUM_TO_COLOR = {
    0: "red",
    1: "green",
    2: "blue",
    3: "yellow",
    4: "purple",
    5: "black",
}

COLOR_TO_NUM = {
    "red": 0,
    "green": 1,
    "blue": 2,
    "yellow": 3,
    "purple": 4,
    "black": 5,
}

GAME_STATUS = {
    0: "resetting blocks",
    1: "waiting for player 2", # player 2 watches for this to make guess
    2: "waiting for robot arm", # player 2 publishes this, robot_arm watches for
    this
    3: "waiting for computer_vision", # robot_arm publishes this, CV watches for
    this
    4: "player 2 wins", # end state 1
    5: "player 2 loses", # end state 2
}

class GameState(Node):
    """
    Node to keep track of game state and pass messages.

    Block diagram, with game status code (sent via /game_status topic)
    or trigger (sent via /submit_code or /guess_check topic):
    """
```



```

def __init__(self, max_guesses: int = 10):
    super().__init__("game_state")

    # Keep track for observability (i.e., logging)
    self.current_game_status: int = 0 # see GAME_STATUS

    # Code related stuff
    self.secret_code: List[int] = [0, 0, 0, 0] # secret code from Player 1
    self.last_guess: List[int] = [0, 0, 0, 0] # last guessed code

    # Guess related stuff
    self.num_guesses: int = 0 # number of current guess
    self.max_guesses: int = max_guesses # maximum number of guesses
    self.num_correct_colors: int = 0 # number of correctly-guessed colors
    self.num_correct_pos: int = 0 # number of correctly-guessed positions

    # Subscriptions
    self.submit_code_sub = self.create_subscription(
        Code, "submit_code", self.handle_code, 10
    )
    self.game_status_sub = self.create_subscription(
        Status, "game_status", self.handle_status, 10
    )

    # Publishers
    self.game_status_pub = self.create_publisher(Status, "game_status", 10)
    self.guess_check_pub = self.create_publisher(GuessCheck, "guess_check", 10)

    # This is here as an example only (not used by this node) - delete later
    self.code_pub = self.create_publisher(Code, "submit_code", 10)

def publish_game_status(self, status: int):
    """
    Publish given status from sender "game_state".
    """
    msg = Status()
    msg.sender = "game_state"
    msg.status = status
    self.game_status_pub.publish(msg)

```

```

def publish_guess_check(self):
    """
    Publish how many colors and positions are correct
    to guess_check topic.
    """
    msg = GuessCheck()
    msg.num_correct_colors = self.num_correct_colors
    msg.num_correct_pos = self.num_correct_pos

    self.get_logger().info(
        f"Player 2 got {self.num_correct_colors} colors and
{self.num_correct_pos} positions correct!"
    )

    # Publish to guess_check so player_2 has feedback
    self.guess_check_pub.publish(msg)

def handle_status(self, msg: Status):
    """
    Subscribe to 'game_status' and keep track of current game status.
    """
    sender = msg.sender
    status = msg.status

    self.current_game_status = status
    self.get_logger().info(
        f"Game status: {status} ({GAME_STATUS[status]}) set by {sender}."
    )

def handle_code(self, msg: Code):
    """
    Store secret code if code message is from player_1.
    If sender is computer_vision, then we check whether
    the guess is correct and keep track of it.

    We don't accept anything from player_2 because
    player_2 only talks to robot_arm, then robot_arm talks
    to computer_vision.
    """
    # If code comes from player_1, keep track of it as secret
    # and publish game status 1 ("waiting for player 2")
    if msg.player_name == "player_1":
        self.get_logger().info("Secret code received from player 1!")

        self.secret_code = msg.code
        self.publish_game_status(1)

```



```

        # If code comes from player 2, just set game_status to 2 (waiting for robot
arm)
    elif msg.player_name == "player_2":
        self.get_logger().info(f"Code {msg.code} received from player 2!")
        self.publish_game_status(2)

    # If code comes from computer_vision, check guess
    # and publish appropriate status and/or feedback for player_2
    elif msg.player_name == "VisionBot":
        self.get_logger().info(f"Code {msg.code} received from
computer_vision")

        self.last_guess = msg.code
        self.num_guesses += 1

        guess_correct = self.check_guess()

        # If guess is correct and we haven't gone over max guess,
        # player 2 wins
        if guess_correct and self.num_guesses <= self.max_guesses:
            self.publish_game_status(4)

            self.get_logger().info("Correct guess...")
            self.get_logger().info("Player 2 wins!")

        elif not guess_correct:
            self.get_logger().info("Incorrect guess...")

            # If guess is not correct and we're at (or above) max,
            # player 2 loses
            if self.num_guesses >= self.max_guesses:
                self.get_logger().info("Player 2 loses!")
                self.publish_game_status(5)

            # Otherwise, send feedback, reset, and go to next round
            else:
                self.get_logger().info("Try again!")

                self.publish_guess_check()

                # Reset blocks and wait 30 seconds for IRL reset
                self.publish_game_status(0)
                time.sleep(30)

            self.publish_game_status(1) # "waiting for player_2"

```

```

def check_guess(self):
    """
    Check guess for number of correct colors and
    number of correct positions.
    """
    # Reset per check
    self.num_correct_colors = 0
    self.num_correct_pos = 0

    for i, c in enumerate(self.last_guess):
        # Check if each guessed color exists in secret code
        if c in self.secret_code:
            self.num_correct_colors += 1

        # Check if each guessed color is in the right place
        if c == self.secret_code[i]:
            self.num_correct_pos += 1

    # For convenience, return True if guess matches secret
    return self.num_correct_colors == 4 and self.num_correct_pos == 4

```

mastermind/src/players/player2.py

```

#!/opt/ros_python/bin/python3

import os
import threading
import time
from typing import List
import rclpy
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from mastermind_interfaces.msg import Code, GuessCheck, Status

from ..game_state.game_state import COLOR_TO_NUM, GAME_STATUS
import os
from openai import OpenAI
from pydantic import BaseModel
from enum import Enum
from typing import List

class Client():
    def __init__(self):
        self.client = OpenAI()
        self.history = []

```

```

def guess(self):
    system_prompt = """
You are a logical reasoning assistant playing MasterMind.
Rules and constraints:
There are exactly 6 possible colors with one each, the secret code length is 4.
The secret uses distinct color only. Every guess you propose must contain 4
distinct number.
Feedback after each guess is given as exact match(A) and color match(B).
For example: 2 Exact Match and 3 Color Match will be represented by 2A3B.
Your job: Analyze guess and feedback history.
Maintain a consistent candidate set under constraint.
Propose the next guess that best narrows the search.
Eliminate impossible numbers.
Output your guess in JSON format.
"""

    user_prompt = """We are playing mastermind with no repetition. Possible
colors: red, green, blue, yellow, purple, black.
You will explain the reasoning process, then give me your guess in this
round."""

    if not len(self.history) == 0:
        user_prompt += "Guess History: "
        for item in self.history:
            user_prompt += f"Guess: {item['guess']}; Feedback:
{item['feedback']}"

    user_prompt += """Example output: {
  \"reasoning\": \"Why this guess reduces the space under the no-duplicate
constraint.\",
  \"guess\": [\"red\", \"green\", \"blue\", \"yellow\"]
}"""

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ]

    class GuessRes(str, Enum):
        red = "red"
        green = "green"
        blue = "blue"
        yellow = "yellow"
        purple = "purple"
        black = "black"

    class ResponseFormat(BaseModel):
        reasoning: str

```

```

        guess: List["GuessRes"]

    response = self.client.responses.parse(
        model="gpt-5-nano",
        input = messages,
        text_format=ResponseFormat
    )

    parsed: ResponseFormat = response.output_parsed

    # guess_list = [g.value for g in parsed.guess]
    return parsed.reasoning, [g.value for g in parsed.guess]

class Player2(Node):
    """
    Player 2 node. This is where we can plug in the ChatGPT node,
    or use it as a standalone player 2.
    """

    def __init__(self):
        super().__init__("player_2")
        self.declare_parameter("mode", "human")

        # Publishers
        self.code_pub = self.create_publisher(Code, "submit_code", 10)

        # For input thread
        self.stop_flag = False

        self.client = None
        self.game_status_sub = self.create_subscription(
            GuessCheck, "guess_check", self.handle_guess, 10
        )

    def handle_guess(self, msg: GuessCheck):
        num_correct_colors = msg.num_correct_colors
        num_correct_pos = msg.num_correct_pos
        if self.mode == 'ai':
            self.client.history[-1] = ({ "guess": self.client.history[-1]['guess'],
            "feedback": f"{num_correct_colors} correct color(s), {num_correct_pos} correct
            position(s)!"})
            self.one_round()
        else:

```

```
        self.get_logger().info(f"Result in this round: \n {num_correct_colors}
correct color(s), {num_correct_pos} correct position(s)\n")
        self.one_round()
```

```
def publish_code(self, code: List[int]):
    """
    Publish guess to /submit_code topic
    """
    msg = Code()
    msg.player_name = "player_2" # player_1, player_2, computer_vision, etc.
    msg.code = code

    self.code_pub.publish(msg)

def check_and_publish_color(self, color_list):
    if len(color_list) != 4:
        self.get_logger().warn("Enter 4 colors!")
        return False
    for c in color_list:
        if c not in COLOR_TO_NUM:
            self.get_logger().warn(f"Unknown color '{c}' -- try again.")
            return False
    allowed = {'blue', 'yellow', 'green', 'red', 'purple', 'black'}
    invalid = [c for c in color_list if c not in allowed]
    if invalid:
        return False
    code = [COLOR_TO_NUM[c] for c in color_list]
    self.publish_code(code)
```

```
def input_loop(self):
    """
    Continuously prompt the user for guesses and publish them.
    """
    # Ask user for input
    user_input = input(
        # "\nEnter your guess (e.g. 'red blue red blue') or 'q' to quit:\n"
        "\nEnter your guess (e.g. 'red blue black green'):\n"
    ).strip()

    color_list = [c.lower() for c in user_input.split()]

    self.check_and_publish_color(color_list)
```

```

def ai_loop(self):
    reasoning, color_list = self.client.guess()
    self.get_logger().info(reasoning)
    self.get_logger().info(f"We are going to guess list {str(color_list)}")
    self.client.history.append({"guess": str(color_list)})
    self.check_and_publish_color(color_list)

def destroy_node(self):
    """
    Ensure the input thread stops on shutdown.
    """
    self.stop_flag = True
    time.sleep(0.2)
    super().destroy_node()

def one_round(self):
    if self.mode == "human":
        self.get_logger().info("Player 2 running in HUMAN mode.")
        self.stop_flag = False
        self.input_loop()

    else:
        self.get_logger().info("Player 2 running in AI mode.")
        self.stop_flag = False
        self.ai_loop()

def run(self):
    self.client = Client()
    self.mode = self.get_parameter("mode").get_parameter_value().string_value
    if self.mode != "human" and self.mode != "ai":
        raise ValueError("Mode must either human or ai")
    executor = MultiThreadedExecutor(num_threads=2)
    executor.add_node(self)

    try:
        threading.Thread(target=executor.spin, daemon=True).start()
        self.one_round()
        while rclpy.ok():
            time.sleep(1)

    finally:
        executor.shutdown()
        self.destroy_node()
        rclpy.shutdown()

```

```

def main():
    rclpy.init()
    node = Player2()

    node.run()

if __name__ == "__main__":
    main()

```

mastermind/src/robot_controller/pick_and_place.py

```

#!/usr/bin/env python3

import os
import subprocess
import threading
import time

import rclpy
from geometry_msgs.msg import Pose
from mastermind_interfaces.msg import Code, Status
from pymoveit2 import MoveIt2
from pymoveit2.gripper_interface import GripperInterface
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from ..game_state.game_state import COLOR_TO_NUM, GAME_STATUS, NUM_TO_COLOR

def make_pose(x, y, z, qx, qy, qz, qw) -> Pose:
    p = Pose()
    p.position.x, p.position.y, p.position.z = x, y, z
    p.orientation.x, p.orientation.y, p.orientation.z, p.orientation.w = qx, qy, qz, qw
    return p

class PickAndPlaceNode(Node):
    def __init__(self):
        super().__init__("pick_and_place")

        # choose task at runtime
        self.declare_parameter("task", "home")
        self.declare_parameter("order", "red blue yellow green")

```

```

# plate height offset (from your spawn)
self.declare_parameter("plate_height_offset", 0.01)

# wait times for gripper actions
self.declare_parameter("close_wait_sec", 1.2)
self.declare_parameter("open_wait_sec", 0.4)

# TODO: Calibrate
# extra release clearance above plate to avoid tilted block clipping
through
self.declare_parameter("place_clearance", 0.001) # 1 cm

# Gazebo world name (spawn uses "empty")
self.declare_parameter("world", "empty")

# Gazebo block center height (from spawn_blocks.sh)
# TABLE_TOP_Z=0.30, BLOCK_SIZE=0.04, BLOCK_DROP=0.01 -> BLOCK_Z = 0.33
self.declare_parameter("gz_block_center_z", 0.33)

self.close_wait_sec = (
    self.get_parameter("close_wait_sec").get_parameter_value().double_value
)
self.open_wait_sec = (
    self.get_parameter("open_wait_sec").get_parameter_value().double_value
)
self.place_clearance = (

self.get_parameter("place_clearance").get_parameter_value().double_value
)
    self.world_name =
self.get_parameter("world").get_parameter_value().string_value
    self.gz_block_center_z = (

self.get_parameter("gz_block_center_z").get_parameter_value().double_value
)

# --- robot / moveit setup ---
self.moveit2 = MoveIt2(
    node=self,
    joint_names=[
        "joint_1",
        "joint_2",
        "joint_3",
        "joint_4",
        "joint_5",
        "joint_6",

```



```

    ],
    base_link_name="base_link",
    end_effector_name="end_effector_link",
    group_name="arm",
)

# ----- world layout -----
table_z = -0.0001
block_size = 0.04

self.table_center_z = table_z - 0.05 / 2.0
self.block_center_z = table_z + block_size / 2.0

block_x = 0.45

# block poses MUST match your spawn_blocks.sh
self.blocks_xyz = [
    (0.33, -0.08, self.block_center_z), # block_1: RED
    (0.45, -0.16, self.block_center_z), # block_2: GREEN
    (0.45, 0.00, self.block_center_z), # block_3: BLUE
    (0.45, -0.08, self.block_center_z), # block_4: YELLOW
    (0.33, 0.0, self.block_center_z), # block_5: PURPLE
    (0.33, -0.16, self.block_center_z), # block_6: BLACK
]

# UPDATED drop spots you already tuned
self.drop_spots = [
    (0.20, 0.21),
    (0.20, 0.28),
    (0.20, 0.35),
    (0.20, 0.42),
]

# TODO: Calibrate IR
self.approach_height = 0.2
self.grasp_height = 0.13

# placing height lifted to match plate being higher/thicker
self.plate_height_offset = (

self.get_parameter("plate_height_offset").get_parameter_value().double_value
)
self.place_height = self.grasp_height + self.plate_height_offset

self.top_down_orientation = (-0.7071, 0.7071, 0.0, 0.0)

# helper poses (based on first block)

```

```

    pick_xyz = self.blocks_xyz[0]
    self.pre_grasp_pose = make_pose(
        pick_xyz[0], pick_xyz[1], self.approach_height,
        *self.top_down_orientation
    )
    self.grasp_pose = make_pose(
        pick_xyz[0], pick_xyz[1], self.grasp_height, *self.top_down_orientation
    )

    target_xyz = self.blocks_xyz[1]

    # place_z uses clearance
    place_z = self.place_height + self.place_clearance
    self.pre_place_pose = make_pose(
        target_xyz[0],
        target_xyz[1],
        self.approach_height,
        *self.top_down_orientation,
    )
    self.place_pose = make_pose(
        target_xyz[0], target_xyz[1], place_z, *self.top_down_orientation
    )

    self.j_home = [0.0] * 6
    self.j_retract = [0.33, 0.02, 2.27, -1.57, -0.84, 1.97]

    self.touch_links = [
        "end_effector_link",
        "right_finger_bottom_link",
        "left_finger_bottom_link",
    ]

    self.block_size = block_size
    self.release_up_dz = 0.03

    self.setup_gripper()

    # Original pickup xyz per color (use same x,y each time)
    self.color_spawn_xyz = {c: self.blocks_xyz[i] for c, i in
        COLOR_TO_NUM.items()}

    # Queue of available MoveIt collision IDs per color
    # Start with original block_1..block_6 IDs
    self.color_pick_queue = {
        "red": ["block_1"],
        "green": ["block_2"],
        "blue": ["block_3"],
    }

```

```

        "yellow": ["block_4"],
        "purple": ["block_5"],
        "black": ["block_6"],
    }

    # Counter to make unique respawned names
    # self.spawn_counter = {c: 1 for c in COLOR_TO_NUM.keys()}

    # detect gz long-flag support (like your spawn script)
    self.gz_use_long_flags = self._gz_has_long_flags()

    # Pub/sub
    self.game_status_pub = self.create_publisher(Status, "game_status", 10)
    self.submit_code_sub = self.create_subscription(
        Code, "submit_code", self.handle_code, 10
    )

def setup_gripper(self):
    """
    Set up gripper interface. This is called once during initialization
    and once every time we reset (for next round)
    """
    # ----- GRIPPER SETUP -----
    action_name = self._auto_find_gripper_action()
    self.get_logger().info(f"Using gripper action: {action_name}")

    try:
        self.gripper = GripperInterface(
            node=self,
            gripper_joint_names=["right_finger_bottom_joint"],
            open_gripper_joint_positions=[0.41],
            closed_gripper_joint_positions=[0.75],
            gripper_command_action_name=action_name,
            ignore_new_calls_while_executing=False,
        )
    except Exception as e:
        self.get_logger().warn(f"GripperInterface initialization failed: {e}")
        self.gripper = None

def reset_blocks(self):
    """
    Reset blocks in memory and MoveIt2.

    The actual sh script that updates the world
    is run by main.py
    """
    self.get_logger().info("Resetting blocks")

```

```

# self.setup_gripper()

self.color_pick_queue = {
    "red": ["block_1"],
    "green": ["block_2"],
    "blue": ["block_3"],
    "yellow": ["block_4"],
    "purple": ["block_5"],
    "black": ["block_6"],
}

# Reset in MoveIt2 (remove collision objects and add them back in)
for i in range(1, 7):
    self.moveit2.remove_collision_object(f"block_{i}")
    time.sleep(0.02)

self.add_scene()

def handle_code(self, msg: Code):
    """
    Callback for submit_code topic subscriber.
    Task:
        - Parse Code message to make sure author isn't player_1
        - Turn code integers to code string, e.g., "red blue red blue"
        - Run task_place_by_color(colors_str) with color string
    """
    player_name = msg.player_name
    code = msg.code

    self.get_logger().info(f"Arm received {code} from {player_name}!")

    if player_name != "player_2":
        return

    self.get_logger().info("Starting pick and place...")

    # Turn numeric code to string and pass to pick and place method
    colors = [NUM_TO_COLOR[c] for c in code]
    colors_str = " ".join(colors)
    threading.Thread(
        target=self.task_place_by_color, args=(colors_str,), daemon=True
    ).start()

def publish_game_status(self, status: int):
    """
    Publish given status from sender 'robot_arm'.
    """

```

```

msg = Status()
msg.sender = "robot_arm"
msg.status = status
self.game_status_pub.publish(msg)

# ----- GRIPPER AUTO-DETECT -----
def _auto_find_gripper_action(self) -> str:
    fallback = "/gen3_lite_2f_gripper_controller/gripper_cmd"
    try:
        acts = self.get_action_names_and_types()
        names = [a[0] for a in acts]
    except Exception:
        return fallback

    preferred = [
        "/gen3_lite_2f_gripper_controller/gripper_cmd",
        "/gripper_controller/gripper_cmd",
        "/robotiq_gripper_controller/gripper_cmd",
    ]
    for p in preferred:
        if p in names:
            return p
    for n in names:
        low = n.lower()
        if "gripper_cmd" in low or ("gripper" in low and "cmd" in low):
            return n
    return fallback

# ----- gz helpers -----
def _gz_has_long_flags(self) -> bool:
    try:
        out = subprocess.check_output(["gz", "service", "--help"], text=True)
        return "--reqtype" in out
    except Exception:
        return True

# ----- utils -----
def move_to_joints(self, joints):
    self.get_logger().info("Move to joints" + str(joints))
    self.moveit2.move_to_configuration(joint_positions=joints)
    self.moveit2.wait_until_executed()

def move_to_pose(self, pose: Pose, cartesian: bool = False) -> bool:
    traj = self.moveit2.plan(
        pose=pose,
        cartesian=cartesian,
        max_step=0.005,
    )

```

```

        cartesian_fraction_threshold=0.90 if cartesian else None,
    )
    if traj is None:
        ec = (
            self.moveit2._planning_result.error_code.val
            if hasattr(self.moveit2, "_planning_result")
            else None
        )
        self.get_logger().warn(f"Planning failed! Error code: {ec}.")
        self.get_logger().error("Planning failed.")
        return False
    self.moveit2.execute(traj)
    self.moveit2.wait_until_executed()
    return True

def open_gripper(self) -> bool:
    if not self.gripper:
        return False
    try:
        self.gripper.open()
        return True
    except Exception as e:
        self.get_logger().error(f"Gripper open failed: {e}")
        return False

def close_gripper(self) -> bool:
    if not self.gripper:
        return False
    try:
        self.gripper.close()
        return True
    except Exception as e:
        self.get_logger().error(f"Gripper close failed: {e}")
        return False

def _micro_lift(self, pose: Pose):
    up = make_pose(
        pose.position.x,
        pose.position.y,
        pose.position.z + self.release_up_dz,
        *self.top_down_orientation,
    )
    _ = self.move_to_pose(up, cartesian=True)

# ----- planning scene -----
def add_scene(self):
    self.get_logger().info("Adding objects to the planning scene.")

```

```

for i, (x, y, z) in enumerate(self.blocks_xyz, start=1):
    self.moveit2.add_collision_box(
        id=f"block_{i}",
        size=(0.04, 0.04, 0.04),
        position=(x, y, z),
        quat_xyzw=(0, 0, 0, 1),
        frame_id="base_link",
    )
    time.sleep(0.05)

self.get_logger().info("Planning scene has been set up.")

# ----- place by color order (supports duplicates + respawn)
-----
def task_place_by_color(self, order_str=None):
    if order_str is None:
        order_str =
self.get_parameter("order").get_parameter_value().string_value

    tokens = [t.strip().lower() for t in order_str.split() if t.strip()]
    if not tokens:
        self.get_logger().warn("No colors given in 'order' param; nothing to
do.")
        return

    self.get_logger().info(f"Placing by color order: {tokens}")
    self.move_to_joints(self.j_retract)
    self.open_gripper()
    time.sleep(self.open_wait_sec)

    drop_idx = 0
    for color in tokens:
        if drop_idx >= len(self.drop_spots):
            self.get_logger().warn("More colors than drop spots; stopping.")
            break

        if color not in self.color_pick_queue:
            self.get_logger().warn(f"Unknown color '{color}', skipping.")
            continue

        if len(self.color_pick_queue[color]) == 0:
            self.get_logger().warn(f"No available '{color}' blocks to pick.")
            continue

        # next available collision object for that color
        block_id = self.color_pick_queue[color].pop(0)

```

```

pick_x, pick_y, _ = self.color_spawn_xyz[color]
drop_x, drop_y = self.drop_spots[drop_idx]

self.get_logger().info(
    f"Placing color '{color}' -> {block_id} to spot {drop_idx+1}"
)

pre_pick = make_pose(
    pick_x, pick_y, self.approach_height, *self.top_down_orientation
)
pick_pose = make_pose(
    pick_x, pick_y, self.grasp_height, *self.top_down_orientation
)
pre_drop = make_pose(
    drop_x, drop_y, self.approach_height, *self.top_down_orientation
)

drop_pose = make_pose(
    drop_x,
    drop_y,
    self.place_height + self.place_clearance,
    *self.top_down_orientation,
)
self.get_logger().info(f"Going to pre pick")
if not self.move_to_pose(pre_pick):
    self.get_logger().error(f"Could not go above {block_id}")
    return

self.get_logger().info(f"Going to pick pose")

if not self.move_to_pose(pick_pose, cartesian=True):
    self.get_logger().error(f"Could not descend to {block_id}")
    return

# squat_pose = make_pose(
#     pick_x, pick_y, self.grasp_height - 0.002,
*self.top_down_orientation
# )
# _ = self.move_to_pose(squat_pose, cartesian=True)
self.get_logger().info(f"Closing gripper")

ok = self.close_gripper()
time.sleep(0.1)
if not ok:
    self.get_logger().warn("Close gripper failed, attaching anyway.")
self.moveit2.attach_collision_object(

```



```

        block_id, "end_effector_link", self.touch_links
    )

    time.sleep(self.close_wait_sec)
    self.get_logger().info(f"Going to pre pick")

    if not self.move_to_pose(pre_pick, cartesian=True):
        self.get_logger().error(f"Could not lift {block_id}")
        return
    self.get_logger().info(f"Going to pre drop")

    if not self.move_to_pose(pre_drop):
        self.get_logger().error(f"Could not move above drop for
{block_id}")
        return
    self.get_logger().info(f"Going to drop place")

    if not self.move_to_pose(drop_pose, cartesian=True):
        self.get_logger().error(f"Could not descend to drop for
{block_id}")
        return

    self.open_gripper()
    time.sleep(self.open_wait_sec)
    self._micro_lift(drop_pose)

    # Detach and REMOVE from MoveIt scene so the placed block stays on
plate
    self.moveit2.detach_collision_object(block_id)
    self.moveit2.remove_collision_object(block_id)
    time.sleep(0.1)

    # Respawn same color on the Gazebo table + add to MoveIt queue
    # self._respawn_color(color)

    _ = self.move_to_pose(pre_drop, cartesian=True)

    drop_idx += 1

    self.move_to_joints(self.j_retract)
    self.get_logger().info("Color-ordered placement complete.")

    # Set game status to 3 so camera can start
    self.publish_game_status(3)

    # ----- shortcuts -----
    def task_home(self):

```

```

        self.move_to_joints(self.j_home)

    def task_retract(self):
        self.move_to_joints(self.j_retract)

    def task_add_scene(self):
        self.add_scene()

```

mastermind/src/vision_model/vision.py

```

#!/usr/bin/env python3
import json
import cv2
import numpy as np
import rclpy
from cv_bridge import CvBridge, CvBridgeError
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String

# --- Import Custom Interfaces ---
try:
    from mastermind_interfaces.msg import Status, Code
except ImportError:
    print("CRITICAL ERROR: Could not import 'mastermind_interfaces'.")
    # Dummy classes for syntax check
    class Status: pass
    class Code: pass

class VisionNode(Node):
    """
    Vision node for Mastermind game.
    Detects 6 possible colors using HSV.
    Strictly requires exactly 4 detected blocks.
    Listens for Status(status=4) -> Publishes Code(player_name, code).
    """

    # Color to Index Mapping
    COLOR_TO_INDEX = {
        "red": 0,
        "green": 1,
        "blue": 2,
        "yellow": 3,
        "purple": 4,
        "black": 5,
    }

```

```
ERROR_INDEX = 255
```

```
def __init__(self) -> None:
    super().__init__("vision_node")
    self.bridge = CvBridge()
    self.latest_image = None
    self.latest_stamp = None
    self.scan_requested = False

    # --- Parameters ---
    self.declare_parameter("image_topic", "/image_raw")
    self.declare_parameter("game_status_topic", "/game_status")
    self.declare_parameter("submit_code_topic", "/submit_code")
    self.declare_parameter("debug_topic",
"mastermind/scanned_guess/debug_text")
    self.declare_parameter("player_name", "VisionBot")
    self.declare_parameter("auto_scan_timer", 0.0)

    image_topic =
self.get_parameter("image_topic").get_parameter_value().string_value
    # game_status_topic =
self.get_parameter("game_status_topic").get_parameter_value().string_value
    self.game_status_pub = self.create_publisher(Status, "game_status", 10)

    self.submit_code_topic =
self.get_parameter("submit_code_topic").get_parameter_value().string_value
    self.player_name =
self.get_parameter("player_name").get_parameter_value().string_value
    self.debug_topic =
self.get_parameter("debug_topic").get_parameter_value().string_value

    # --- HSV Color Ranges ---
    self.color_ranges = {
        'red': ([0, 100, 100], [10, 255, 255]),
        'red2': ([170, 100, 100], [180, 255, 255]),
        'green': ([40, 90, 100], [90, 255, 255]),
        'blue': ([100, 200, 100], [130, 255, 255]),
        'yellow': ([10, 110, 100], [35, 255, 255]),
        'purple': ([130, 100, 100], [160, 200, 255]),
        'black': ([0, 0, 0], [120, 130, 60])
    }

    # --- ROS Setup ---
    self.image_subscription = self.create_subscription(
        Image, image_topic, self._image_callback, 10
    )
```

```

# 1. Subscribe to game_status (Custom Message)
self.status_subscription = self.create_subscription(
    Status,
    "game_status",
    self._status_callback,
    10
)

# 2. Publish Code (Custom Message)
self.code_pub = self.create_publisher(Code, self.submit_code_topic, 10)

self.get_logger().info(f"Vision Node ready. Waiting for Status=4.")

def _image_callback(self, msg: Image) -> None:
    try:
        self.latest_image = self.bridge.imgmsg_to_cv2(msg,
desired_encoding="bgr8")
        self.latest_stamp = msg.header.stamp
    except CvBridgeError as exc:
        self.get_logger().error(f"Failed to convert image: {exc}")
        return

    if self.scan_requested:
        self.scan_requested = False
        self._process_frame()

def _status_callback(self, msg: Status) -> None:
    """
    Callback for /game_status.
    Wait for status == 4 to trigger scan.
    """
    if msg.status == 3:
        if self.latest_image is not None:
            self.get_logger().info(f"Received Status 3 from '{msg.sender}'.
Scanning...")
            self._process_frame()
        else:
            self.get_logger().warn("Received Status 3, but no image yet.
Queuing scan.")
            self.scan_requested = True

def _process_frame(self) -> None:
    self.get_logger().info("process frame")
    if self.latest_image is None: return

```

```

image = self._pre_process_img(self.latest_image.copy())

try:
    ok = cv2.imwrite("./debug.png", image)
    if not ok:
        self.get_logger().warn("Failed to write debug image to
./debug.png")
except Exception as e:
    self.get_logger().error(f"Exception while saving debug image: {e}")

height, width, _ = image.shape

# --- 1. ROI Cropping (Updated for closer camera) ---
roi_y_start = int(height * 0.33)
roi_y_end   = int(height * 0.60)
roi_x_start = int(width * 0.18)
roi_x_end   = int(width * 0.82)
roi_img = image[roi_y_start:roi_y_end, roi_x_start:roi_x_end]

hsv = cv2.cvtColor(roi_img, cv2.COLOR_BGR2HSV)
detected_blocks = []

# --- 2. Color Detection ---
for color_name, (lower, upper) in self.color_ranges.items():
    if color_name == 'red':
        l2, u2 = self.color_ranges['red2']
        mask = cv2.inRange(hsv, np.array(lower), np.array(upper)) + \
            cv2.inRange(hsv, np.array(l2), np.array(u2))
    elif color_name == 'red2':
        continue
    else:
        mask = cv2.inRange(hsv, np.array(lower), np.array(upper))

kernel = np.ones((5, 5), np.uint8)

mask = cv2.erode(mask, kernel, iterations=3)
mask = cv2.dilate(mask, kernel, iterations=4)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

for c in contours:
    # Increased area limit for closer camera view
    if 800 < cv2.contourArea(c) < 150000:

```

```

        M = cv2.moments(c)
        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            detected_blocks.append((cx, color_name))

# --- 3. Strict Validation ---
count = len(detected_blocks)

# If NOT exactly 4 blocks, return ERROR and do NOT publish Code
if count != 4:
    self.get_logger().error(f"DETECTION ERROR: Found {count} blocks.
Expected exactly 4.")

# --- 4. Sorting and Output (Only if count == 4) ---
detected_blocks.sort(key=lambda x: x[0])

result_labels = [b[1] for b in detected_blocks]
result_indices = [self.COLOR_TO_INDEX.get(lbl, 0) for lbl in result_labels]

self.get_logger().info(f"Success: {result_labels} -> {result_indices}")

# --- Publish Code Message ---
msg = Code()
msg.player_name = self.player_name
msg.code = result_indices

self.code_pub.publish(msg)

def _pre_process_img(self, img, exp_factor=1.2, sat_factor=1.5):

    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV).astype(np.float32)
    h, s, v = cv2.split(hsv)

    s *= sat_factor

    v = v * exp_factor
    s = np.clip(s, 0, 255)
    v = np.clip(v, 0, 255)

    hsv_merged = cv2.merge([h, s, v]).astype(np.uint8)
    out = cv2.cvtColor(hsv_merged, cv2.COLOR_HSV2BGR)
    return out

```